

UNIVERSIDAD
NACIONAL
DE COLOMBIA

Programación Orientada a Objetos 2026-2

Actividad 3: Lógica de programación y programación orientada a objetos

Facultad de Minas

Universidad Nacional de Colombia Sede Medellín

Docente: Walter Hugo Arboleda Mazo

Jerónimo Berrio García

@jeberriog.unal.edu.co

UNIVERSIDAD NACIONAL DE COLOMBIA

Contenido

Ejercicio 8.2 Pag. 483 (Clase Notas) 3
Ejercicio 8.3 Figura Geométrica 9
Url repositorio 19
https://github.com/jeroberrig/trabajo3-POO-2026-2 19

UNIVERSIDAD NACIONAL DE COLOMBIA

Ejercicio 8.2 Pag. 483 (Clase Notas)

```
import tkinter as tk
from tkinter import messagebox
import math

class Notas:
    """
    Clase que define una lista de notas numéricas y realiza cálculos estadísticos.
    """
    def __init__(self, lista_notas):
        self.lista_notas = lista_notas

    def calcular_promedio(self):
        suma = sum(self.lista_notas)
        return suma / len(self.lista_notas)

    def calcular_desviacion(self):
        prom = self.calcular_promedio()
        suma = sum(math.pow(nota - prom, 2) for nota in self.lista_notas)
        return math.sqrt(suma / len(self.lista_notas))

    def calcular_menor(self):
        return min(self.lista_notas)

    def calcular_mayor(self):
        return max(self.lista_notas)


class VentanaPrincipal:
    """
    Clase que define la interfaz gráfica para ingresar y calcular operaciones sobre notas.
    """
    def __init__(self, root):
        self.root = root
        self.root.title("Notas")
        self.root.geometry("280x380")
        self.root.resizable(False, False)

        self.inicio()
```

UNIVERSIDAD NACIONAL DE COLOMBIA

```
def inicio(self):
    # Etiquetas y campos de texto para las notas
    tk.Label(self.root, text="Nota 1:").place(x=20, y=20, width=80, height=23)
    self.campo_nota1 = tk.Entry(self.root)
    self.campo_nota1.place(x=105, y=20, width=135, height=23)

    tk.Label(self.root, text="Nota 2:").place(x=20, y=50, width=80, height=23)
    self.campo_nota2 = tk.Entry(self.root)
    self.campo_nota2.place(x=105, y=50, width=135, height=23)

    tk.Label(self.root, text="Nota 3:").place(x=20, y=80, width=80, height=23)
    self.campo_nota3 = tk.Entry(self.root)
    self.campo_nota3.place(x=105, y=80, width=135, height=23)

    tk.Label(self.root, text="Nota 4:").place(x=20, y=110, width=80, height=23)
    self.campo_nota4 = tk.Entry(self.root)
    self.campo_nota4.place(x=105, y=110, width=135, height=23)

    tk.Label(self.root, text="Nota 5:").place(x=20, y=140, width=80, height=23)
    self.campo_nota5 = tk.Entry(self.root)
    self.campo_nota5.place(x=105, y=140, width=135, height=23)

    # Botones
    self.btn_calcular = tk.Button(self.root, text="Calcular", command=self.calcular_accion)
    self.btn_calcular.place(x=20, y=170, width=100, height=23)

    self.btn_limpiar = tk.Button(self.root, text="Limpiar", command=self.limpiar_accion)
    self.btn_limpiar.place(x=125, y=170, width=80, height=23)

    # Etiquetas de resultados
    self.lbl_promedio = tk.Label(self.root, text="Promedio = ", anchor="w")
    self.lbl_promedio.place(x=20, y=210, width=240, height=23)

    self.lbl_desviacion = tk.Label(self.root, text="Desviación = ", anchor="w")
    self.lbl_desviacion.place(x=20, y=240, width=240, height=23)

    self.lbl_mayor = tk.Label(self.root, text="Nota mayor = ", anchor="w")
    self.lbl_mayor.place(x=20, y=270, width=240, height=23)

    self.lbl_menor = tk.Label(self.root, text="Nota menor = ", anchor="w")
    self.lbl_menor.place(x=20, y=300, width=240, height=23)
```

```
    def calcular_accion(self):
        # Obtener los textos de los campos
        textos_notas = [
            self.campo_nota1.get(),
            self.campo_nota2.get(),
            self.campo_nota3.get(),
            self.campo_nota4.get(),
            self.campo_nota5.get()
        ]

        # Ejercicio propuesto 2: Validar que ninguna nota esté vacía
        for texto in textos_notas:
            if texto.strip() == "":
                messagebox.showwarning("Alerta", "Es obligatorio ingresar las cinco notas. Por favor llene
todos los campos.")
                return

        # Ejercicio propuesto 1: Validar que los datos ingresados sean numéricos
        try:
            valores_notas = [float(texto) for texto in textos_notas]
        except ValueError:
            messagebox.showerror("Error de formato", "Se han ingresado datos que no son numéricos.
Por favor ingrese solo números.")
            return

        # Si todo es correcto, se procede a calcular
        notas = Notas(valores_notas)

        promedio = notas.calcular_promedio()
        desviacion = notas.calcular_desviacion()
        mayor = notas.calcular_mayor()
        menor = notas.calcular_menor()

        # Actualizar las etiquetas con los resultados formateados a 2 decimales
        self.lbl_promedio.config(text=f"Promedio = {promedio:.2f}")
        self.lbl_desviacion.config(text=f"Desviación estándar = {desviacion:.2f}")
        self.lbl_mayor.config(text=f"Valor mayor = {mayor:.2f}")
        self.lbl_menor.config(text=f"Valor menor = {menor:.2f}")

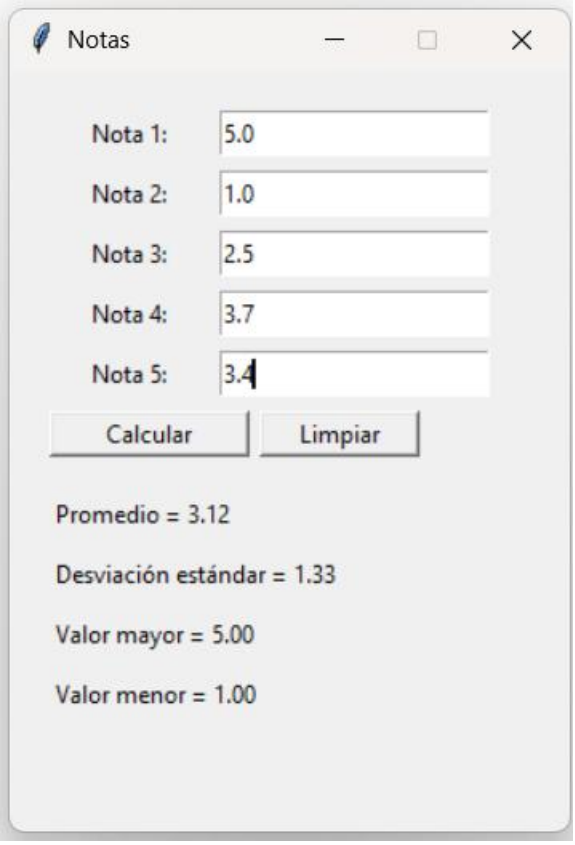
    def limpiar_accion(self):
        # Limpiar los campos de texto
        self.campo_nota1.delete(0, tk.END)
        self.campo_nota2.delete(0, tk.END)
        self.campo_nota3.delete(0, tk.END)
        self.campo_nota4.delete(0, tk.END)
        self.campo_nota5.delete(0, tk.END)
```

UNIVERSIDAD NACIONAL DE COLOMBIA

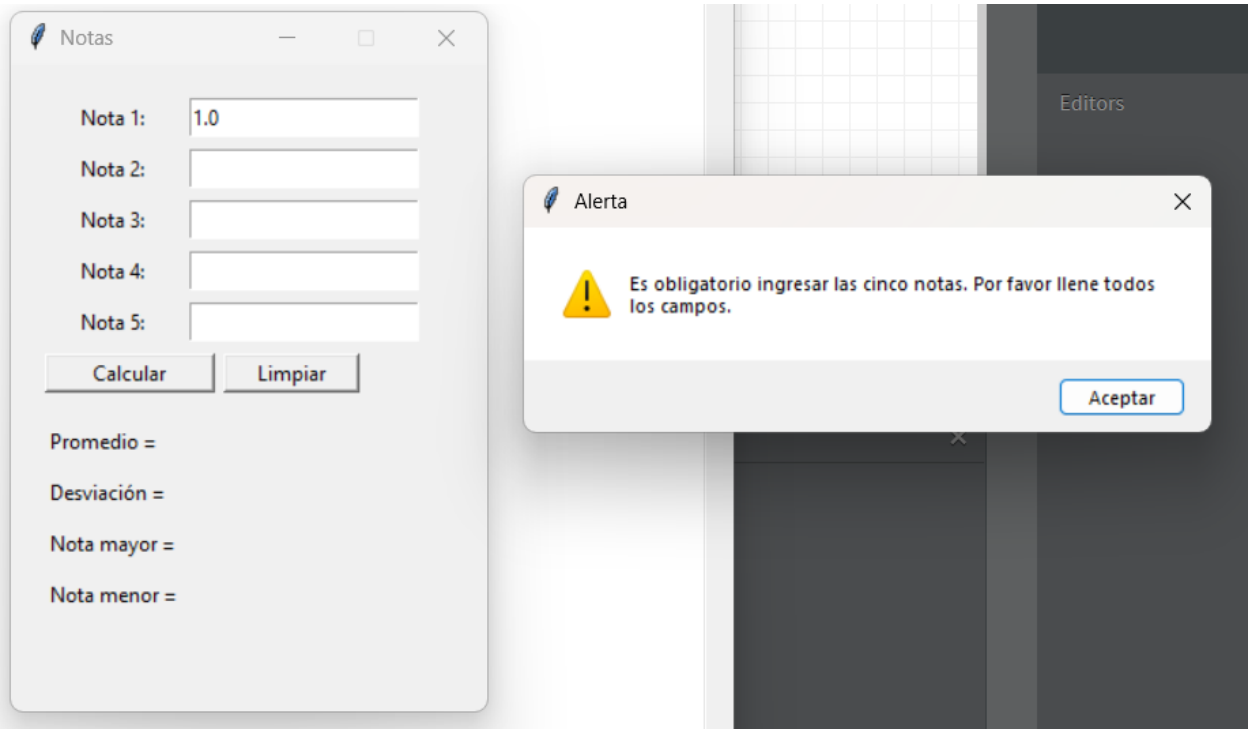
```
        # Restablecer etiquetas
        self.lbl_promedio.config(text="Promedio = ")
        self.lbl_desviacion.config(text="Desviación = ")
        self.lbl_mayor.config(text="Nota mayor = ")
        self.lbl_menor.config(text="Nota menor = ")

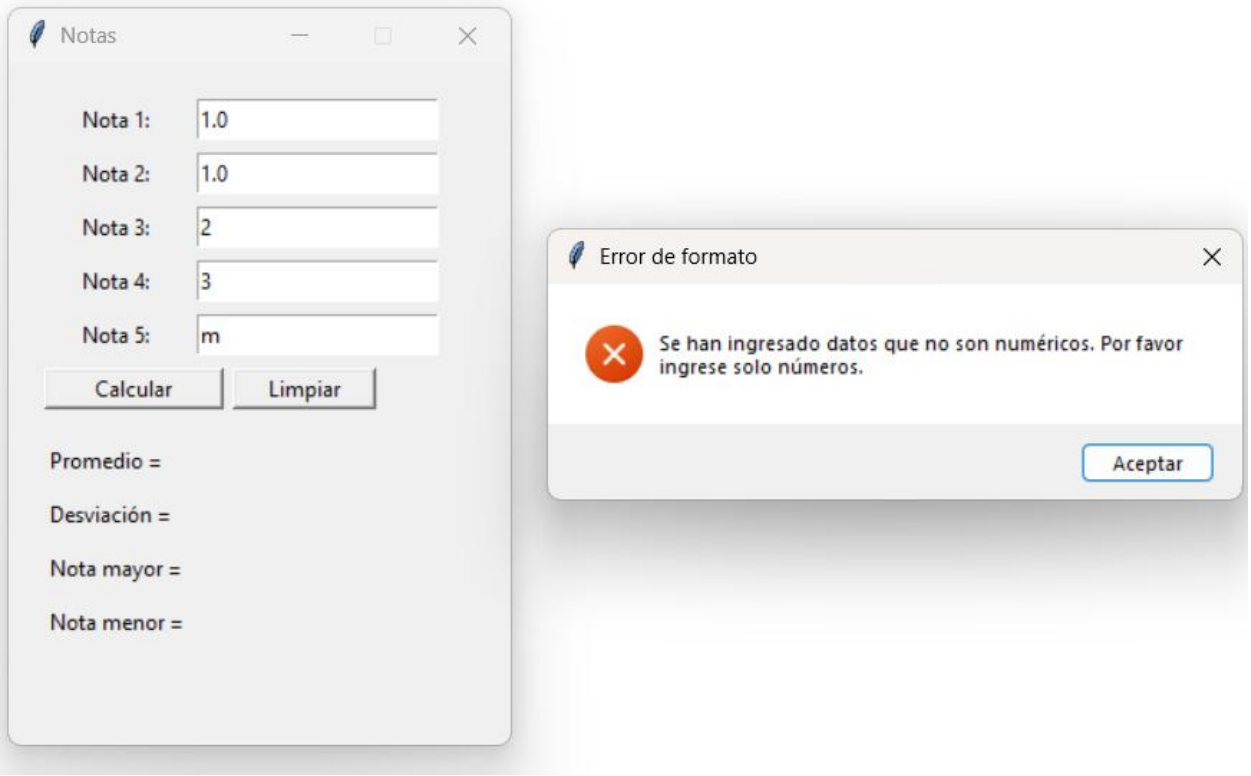

if __name__ == "__main__":
    # Clase Principal: Punto de entrada al programa
    root = tk.Tk()
    app = VentanaPrincipal(root)
    root.mainloop()
```

Interfaz:



UNIVERSIDAD
NACIONAL
DE COLOMBIA





UNIVERSIDAD NACIONAL DE COLOMBIA

Diagrama de clase:

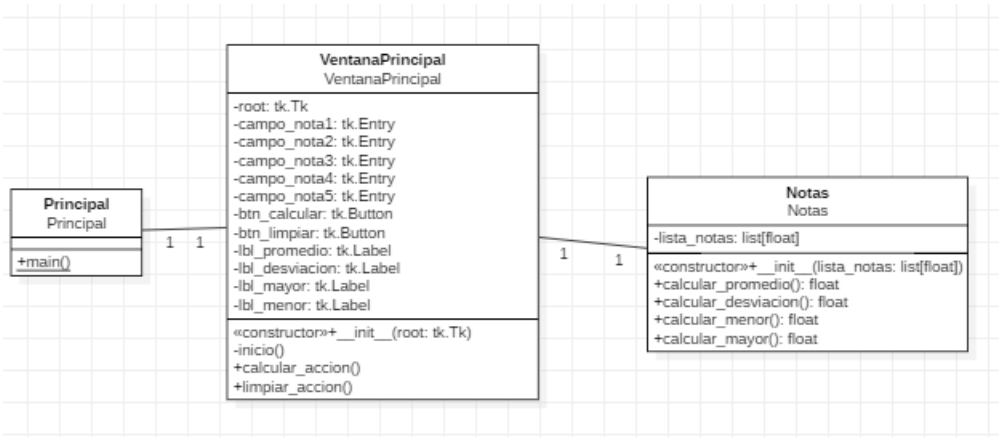


Diagrama de objetos

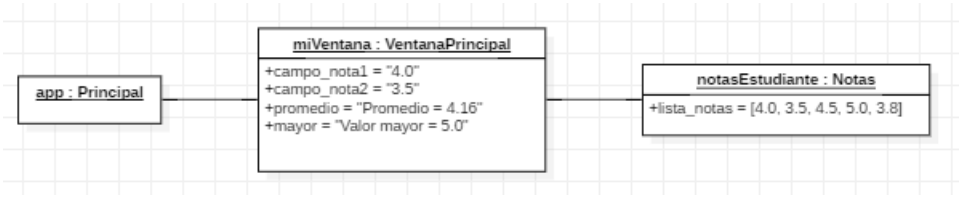
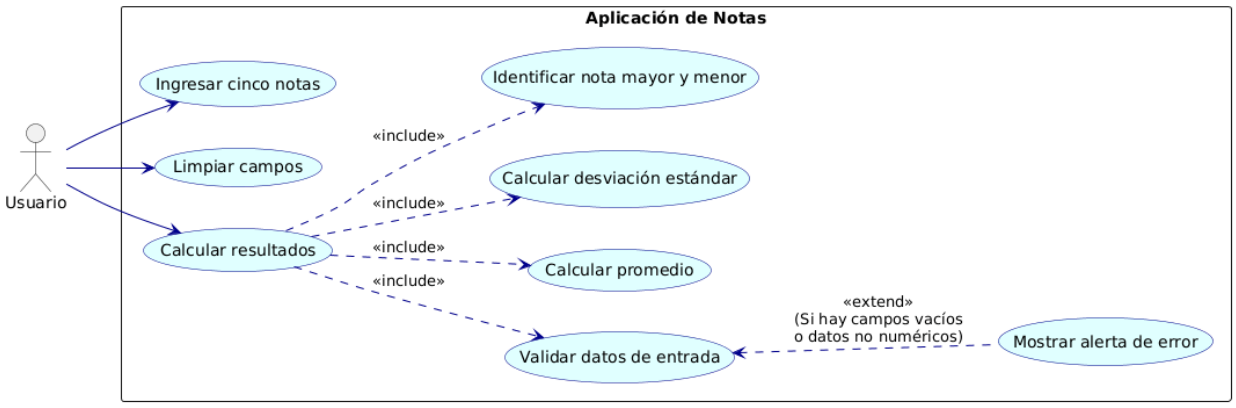


Diagrama de usos:



UNIVERSIDAD NACIONAL DE COLOMBIA

Ejercicio 8.3 Figura Geométrica

```
import math
import tkinter as tk
from tkinter import messagebox

# LÓGICA DE NEGOCIO (CLASES GEOMÉTRICAS)

class FiguraGeometrica:
    def __init__(self):
        self._volumen = 0.0
        self._superficie = 0.0

    def set_volumen(self, volumen):
        self._volumen = volumen

    def set_superficie(self, superficie):
        self._superficie = superficie

    def get_volumen(self):
        return self._volumen

    def get_superficie(self):
        return self._superficie


class Cilindro(FiguraGeometrica):
    def __init__(self, radio, altura):
        super().__init__()
        self.radio = radio
        self.altura = altura
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())

    def calcular_volumen(self):
        return math.pi * self.altura * (self.radio ** 2)

    def calcular_superficie(self):
        area_lado_a = 2.0 * math.pi * self.radio * self.altura
        area_lado_b = 2.0 * math.pi * (self.radio ** 2)
        return area_lado_a + area_lado_b
```

```
class Esfera(FiguraGeometrica):
    def __init__(self, radio):
        super().__init__()
        self.radio = radio
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())

    def calcular_volumen(self):
        return (4.0 / 3.0) * math.pi * (self.radio ** 3)

    def calcular_superficie(self):
        return 4.0 * math.pi * (self.radio ** 2)


class Piramide(FiguraGeometrica):
    def __init__(self, base, altura, apotema):
        super().__init__()
        self.base = base
        self.altura = altura
        self.apotema = apotema
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())

    def calcular_volumen(self):
        return ((self.base ** 2) * self.altura) / 3.0

    def calcular_superficie(self):
        area_base = self.base ** 2
        area_lado = 2.0 * self.base * self.apotema
        return area_base + area_lado

# Nuevas figuras agregadas
class Cubo(FiguraGeometrica):
    def __init__(self, lado):
        super().__init__()
        self.lado = lado
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())

    def calcular_volumen(self):
        return self.lado ** 3

    def calcular_superficie(self):
        return 6.0 * (self.lado ** 2)
```

```
class Prisma(FiguraGeometrica):
    def __init__(self, base, altura, profundidad):
        super().__init__()
        self.base = base
        self.altura = altura
        self.profundidad = profundidad
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())

    def calcular_volumen(self):
        return self.base * self.altura * self.profundidad

    def calcular_superficie(self):
        return 2.0 * (self.base * self.altura + self.base * self.profundidad + self.altura * self.profundidad)

#Dibujo figuras

ANCHO_DIBUJO = 240
ALTO_DIBUJO = 170

# Paleta de colores
COLOR_CLARO = "#9dc3e6"
COLOR_MEDIO = "#5b9bd5"
COLOR_OSCURO = "#2e75b6"
COLOR_BORDE = "#1f4e79"
COLOR_COTA = "#c00000"

def dibujar_cilindro(c):
    # Base inferior y cuerpo
    c.create_oval(80, 120, 160, 140, fill=COLOR_MEDIO, outline=COLOR_BORDE)
    c.create_rectangle(80, 30, 160, 130, fill=COLOR_MEDIO, outline="")
    c.create_line(80, 30, 80, 130, fill=COLOR_BORDE, width=1)
    c.create_line(160, 30, 160, 130, fill=COLOR_BORDE, width=1)
    # Tapa superior
    c.create_oval(80, 20, 160, 40, fill=COLOR_CLARO, outline=COLOR_BORDE)
    # Radio
    c.create_line(120, 30, 160, 30, fill=COLOR_COTA, width=2)
    c.create_oval(118, 28, 122, 32, fill=COLOR_COTA, outline=COLOR_COTA)
    c.create_text(140, 46, text="r", fill=COLOR_COTA, font=("Arial", 10, "bold"))
    # Altura
    c.create_line(182, 30, 182, 130, fill=COLOR_COTA, arrow=tk.BOTH)
    c.create_text(196, 80, text="h", fill=COLOR_COTA, font=("Arial", 10, "bold"))
```

```
def dibujar_esfera(c):
    cx, cy, r = 120, 85, 60
    c.create_oval(cx - r, cy - r, cx + r, cy + r, fill=COLOR_MEDIO, outline=COLOR_BORDE)
    # Brillo
    c.create_oval(cx - 38, cy - 42, cx - 8, cy - 18, fill=COLOR_CLARO, outline="")
    # Ecuador
    c.create_oval(cx - r, cy - 14, cx + r, cy + 14, outline=COLOR_BORDE, dash=(4, 3))
    # Radio
    c.create_line(cx, cy, cx + r, cy, fill=COLOR_COTA, width=2)
    c.create_oval(cx - 3, cy - 3, cx + 3, cy + 3, fill=COLOR_COTA, outline=COLOR_COTA)
    c.create_text(cx + r + 12, cy, text="r", fill=COLOR_COTA, font=("Arial", 10, "bold"))


def dibujar_piramide(c):
    apice = (137, 18)
    fl, fr, br, bl = (70, 130), (170, 130), (205, 105), (105, 105)
    # Base
    c.create_polygon(*fl, *fr, *br, *bl, fill=COLOR_CLARO, outline=COLOR_BORDE)
    # Aristas ocultas
    c.create_line(*bl, *apice, fill=COLOR_BORDE, dash=(4, 3))
    c.create_line(*bl, *br, fill=COLOR_BORDE, dash=(4, 3))
    c.create_line(*bl, *fl, fill=COLOR_BORDE, dash=(4, 3))
    # Caras visibles
    c.create_polygon(*apice, *fl, *fr, fill=COLOR_MEDIO, outline=COLOR_BORDE)
    c.create_polygon(*apice, *fr, *br, fill=COLOR_OSCURO, outline=COLOR_BORDE)
    # Altura (centro de la base hasta el vértice)
    c.create_line(137, 18, 137, 118, fill=COLOR_COTA, dash=(4, 3), width=2)
    c.create_text(147, 66, text="h", fill=COLOR_COTA, font=("Arial", 10, "bold"))
    # Apotema (vértice hasta el punto medio del lado frontal)
    c.create_line(137, 18, 120, 130, fill=COLOR_COTA, width=2)
    c.create_text(107, 75, text="ap", fill=COLOR_COTA, font=("Arial", 10, "bold"))
    # Base
    c.create_text(120, 145, text="b", fill=COLOR_COTA, font=("Arial", 10, "bold"))

def dibujar_cubo(c):
    # Cara frontal
    c.create_rectangle(60, 60, 140, 140, fill=COLOR_MEDIO, outline=COLOR_BORDE)
    # Cara superior
    c.create_polygon(60, 60, 100, 20, 180, 20, 140, 60, fill=COLOR_CLARO, outline=COLOR_BORDE)
    # Cara lateral
    c.create_polygon(140, 60, 180, 20, 180, 100, 140, 140, fill=COLOR_OSCURO,
outline=COLOR_BORDE)
    # Lado
    c.create_text(100, 153, text="l", fill=COLOR_COTA, font=("Arial", 10, "bold"))
    c.create_text(48, 100, text="l", fill=COLOR_COTA, font=("Arial", 10, "bold"))
```

```
def dibujar_prisma(c):
    # Cara frontal
    c.create_rectangle(40, 60, 150, 140, fill=COLOR_MEDIO, outline=COLOR_BORDE)
    # Cara superior
    c.create_polygon(40, 60, 90, 25, 200, 25, 150, 60, fill=COLOR_CLARO, outline=COLOR_BORDE)
    # Cara lateral
    c.create_polygon(150, 60, 200, 25, 200, 105, 150, 140, fill=COLOR_OSCURO,
outline=COLOR_BORDE)
    # Cotas
    c.create_text(95, 153, text="base", fill=COLOR_COTA, font=("Arial", 10, "bold"))
    c.create_text(22, 100, text="altura", fill=COLOR_COTA, font=("Arial", 10, "bold"))
    c.create_text(205, 125, text="prof.", fill=COLOR_COTA, font=("Arial", 10, "bold"))


# INTERFAZ GRÁFICA (GUI)

class VentanaFigura(tk.Toplevel):
    """Clase base para las ventanas de figuras, maneja el dibujo de la figura"""
    def cargar_imagen(self, dibujar):
        lienzo = tk.Canvas(self, width=ANCHO_DIBUJO, height=ALTO_DIBUJO,
                           bg="white", highlightthickness=1,
                           highlightbackground="gray")
        lienzo.pack(pady=10)
        dibujar(lienzo)

class VentanaCilindro(VentanaFigura):
    def __init__(self, master):
        super().__init__(master)
        self.title("Cilindro")
        self.geometry("300x450")
        self.resizable(False, False)

        tk.Label(self, text="Radio (cms):").pack(pady=5)
        self.campo_radio = tk.Entry(self)
        self.campo_radio.pack()

        tk.Label(self, text="Altura (cms):").pack(pady=5)
        self.campo_altura = tk.Entry(self)
        self.campo_altura.pack()

        tk.Button(self, text="Calcular", command=self.calcular).pack(pady=10)

        self.lbl_volumen = tk.Label(self, text="Volumen (cm3):")
        self.lbl_volumen.pack()

        self.lbl_superficie = tk.Label(self, text="Superficie (cm2):")
        self.lbl_superficie.pack()

        self.cargar_imagen(dibujar_cilindro)
```

```
    def calcular(self):
        try:
            r = float(self.campo_radio.get())
            a = float(self.campo_altura.get())
            cilindro = Cilindro(r, a)
            self.lbl_volumen.config(text=f"Volumen (cm3): {cilindro.get_volumen():.2f}")
            self.lbl_superficie.config(text=f"Superficie (cm2): {cilindro.get_superficie():.2f}")
        except ValueError:
            messagebox.showerror("Error", "Campo nulo o error en formato de número")


class VentanaEsfera(VentanaFigura):
    def __init__(self, master):
        super().__init__(master)
        self.title("Esfera")
        self.geometry("300x400")
        self.resizable(False, False)

        tk.Label(self, text="Radio (cms):").pack(pady=5)
        self.campo_radio = tk.Entry(self)
        self.campo_radio.pack()

        tk.Button(self, text="Calcular", command=self.calcular).pack(pady=10)

        self.lbl_volumen = tk.Label(self, text="Volumen (cm3):")
        self.lbl_volumen.pack()

        self.lbl_superficie = tk.Label(self, text="Superficie (cm2):")
        self.lbl_superficie.pack()

        self.cargar_imagen(dibujar_esfera)

    def calcular(self):
        try:
            r = float(self.campo_radio.get())
            esfera = Esfera(r)
            self.lbl_volumen.config(text=f"Volumen (cm3): {esfera.get_volumen():.2f}")
            self.lbl_superficie.config(text=f"Superficie (cm2): {esfera.get_superficie():.2f}")
        except ValueError:
            messagebox.showerror("Error", "Campo nulo o error en formato de número")
```

```
class VentanaPiramide(VentanaFigura):
    def __init__(self, master):
        super().__init__(master)
        self.title("Pirámide")
        self.geometry("300x500")
        self.resizable(False, False)

        tk.Label(self, text="Base (cms):").pack(pady=2)
        self.campo_base = tk.Entry(self)
        self.campo_base.pack()

        tk.Label(self, text="Altura (cms):").pack(pady=2)
        self.campo_altura = tk.Entry(self)
        self.campo_altura.pack()

        tk.Label(self, text="Apotema (cms):").pack(pady=2)
        self.campo_apotema = tk.Entry(self)
        self.campo_apotema.pack()

        tk.Button(self, text="Calcular", command=self.calcular).pack(pady=10)

        self.lbl_volumen = tk.Label(self, text="Volumen (cm3):")
        self.lbl_volumen.pack()

        self.lbl_superficie = tk.Label(self, text="Superficie (cm2):")
        self.lbl_superficie.pack()

        self.cargar_imagen(dibujar_piramide)

    def calcular(self):
        try:
            b = float(self.campo_base.get())
            a = float(self.campo_altura.get())
            ap = float(self.campo_apotema.get())
            piramide = Piramide(b, a, ap)
            self.lbl_volumen.config(text=f"Volumen (cm3): {piramide.get_volumen():.2f}")
            self.lbl_superficie.config(text=f"Superficie (cm2): {piramide.get_superficie():.2f}")
        except ValueError:
            messagebox.showerror("Error", "Campo nulo o error en formato de número")

# Ventanas de las nuevas figuras
class VentanaCubo(VentanaFigura):
    def __init__(self, master):
        super().__init__(master)
        self.title("Cubo")
        self.geometry("300x400")
        self.resizable(False, False)
```

```
        tk.Label(self, text="Lado (cms):").pack(pady=5)
        self.campo_lado = tk.Entry(self)
        self.campo_lado.pack()

        tk.Button(self, text="Calcular", command=self.calcular).pack(pady=10)

        self.lbl_volumen = tk.Label(self, text="Volumen (cm3):")
        self.lbl_volumen.pack()

        self.lbl_superficie = tk.Label(self, text="Superficie (cm2):")
        self.lbl_superficie.pack()

        self.cargar_imagen(dibujar_cubo)

    def calcular(self):
        try:
            l = float(self.campo_lado.get())
            cubo = Cubo(l)
            self.lbl_volumen.config(text=f"Volumen (cm3): {cubo.get_volumen():.2f}")
            self.lbl_superficie.config(text=f"Superficie (cm2): {cubo.get_superficie():.2f}")
        except ValueError:
            messagebox.showerror("Error", "Campo nulo o error en formato de número")

class VentanaPrisma(VentanaFigura):
    def __init__(self, master):
        super().__init__(master)
        self.title("Prisma Rectangular")
        self.geometry("300x500")
        self.resizable(False, False)

        tk.Label(self, text="Base (cms):").pack(pady=2)
        self.campo_base = tk.Entry(self)
        self.campo_base.pack()

        tk.Label(self, text="Altura (cms):").pack(pady=2)
        self.campo_altura = tk.Entry(self)
        self.campo_altura.pack()

        tk.Label(self, text="Profundidad (cms):").pack(pady=2)
        self.campo_profundidad = tk.Entry(self)
        self.campo_profundidad.pack()
```

```
        tk.Button(self, text="Calcular", command=self.calcular).pack(pady=10)

        self.lbl_volumen = tk.Label(self, text="Volumen (cm3):")
        self.lbl_volumen.pack()

        self.lbl_superficie = tk.Label(self, text="Superficie (cm2):")
        self.lbl_superficie.pack()

        self.cargar_imagen(dibujar_prisma)

    def calcular(self):
        try:
            b = float(self.campo_base.get())
            a = float(self.campo_altura.get())
            p = float(self.campo_profundidad.get())
            prisma = Prisma(b, a, p)
            self.lbl_volumen.config(text=f"Volumen (cm3): {prisma.get_volumen():.2f}")
            self.lbl_superficie.config(text=f"Superficie (cm2): {prisma.get_superficie():.2f}")
        except ValueError:
            messagebox.showerror("Error", "Campo nulo o error en formato de número")

class VentanaPrincipal(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Figuras")
        self.geometry("500x120")
        self.resizable(False, False)

        frame = tk.Frame(self)
        frame.pack(expand=True)

        tk.Button(frame, text="Cilindro", command=lambda: VentanaCilindro(self)).pack(side=tk.LEFT,
padx=5)
        tk.Button(frame, text="Esfera", command=lambda: VentanaEsfera(self)).pack(side=tk.LEFT,
padx=5)
        tk.Button(frame, text="Pirámide", command=lambda:
VentanaPiramide(self)).pack(side=tk.LEFT, padx=5)
        tk.Button(frame, text="Cubo", command=lambda: VentanaCubo(self)).pack(side=tk.LEFT,
padx=5)
        tk.Button(frame, text="Prisma", command=lambda: VentanaPrisma(self)).pack(side=tk.LEFT,
padx=5)


if __name__ == "__main__":
    app = VentanaPrincipal()
    app.mainloop()
```

UNIVERSIDAD NACIONAL DE COLOMBIA

Interfaz:

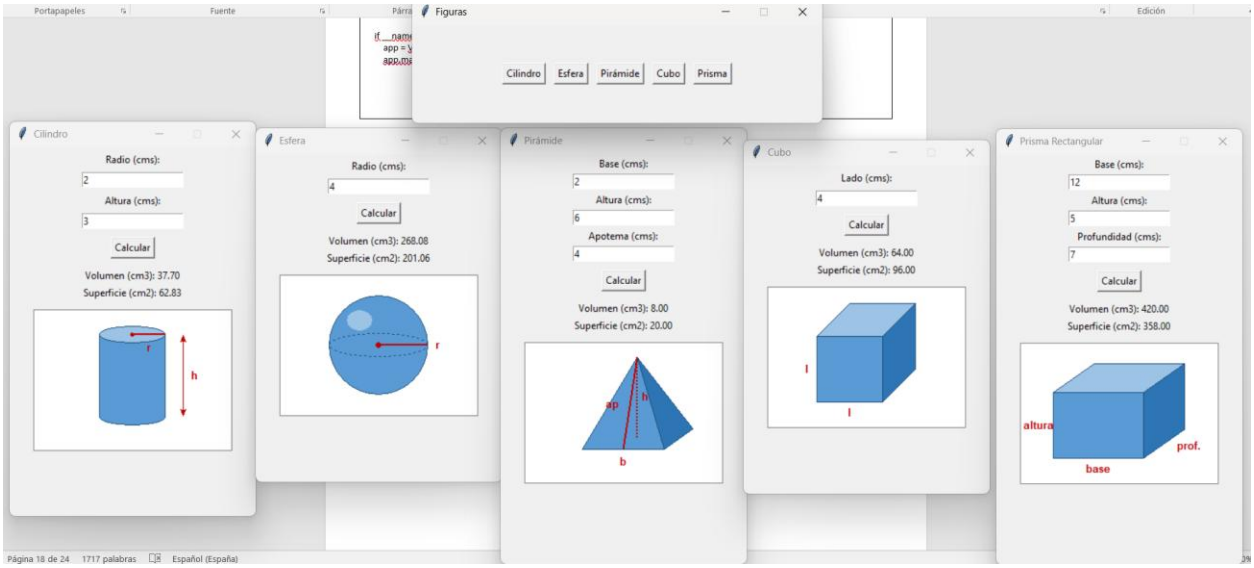
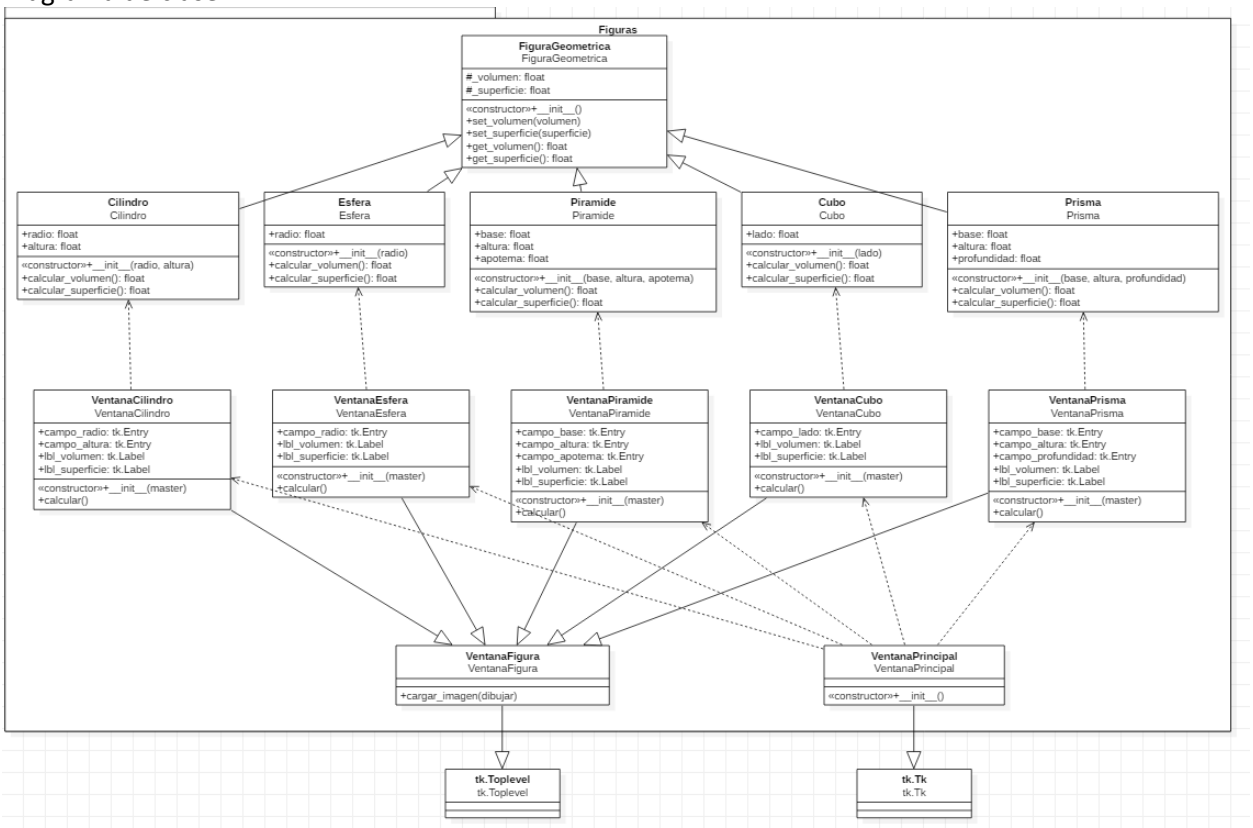


Diagrama de clase



UNIVERSIDAD NACIONAL DE COLOMBIA

Diagrama de objetos:

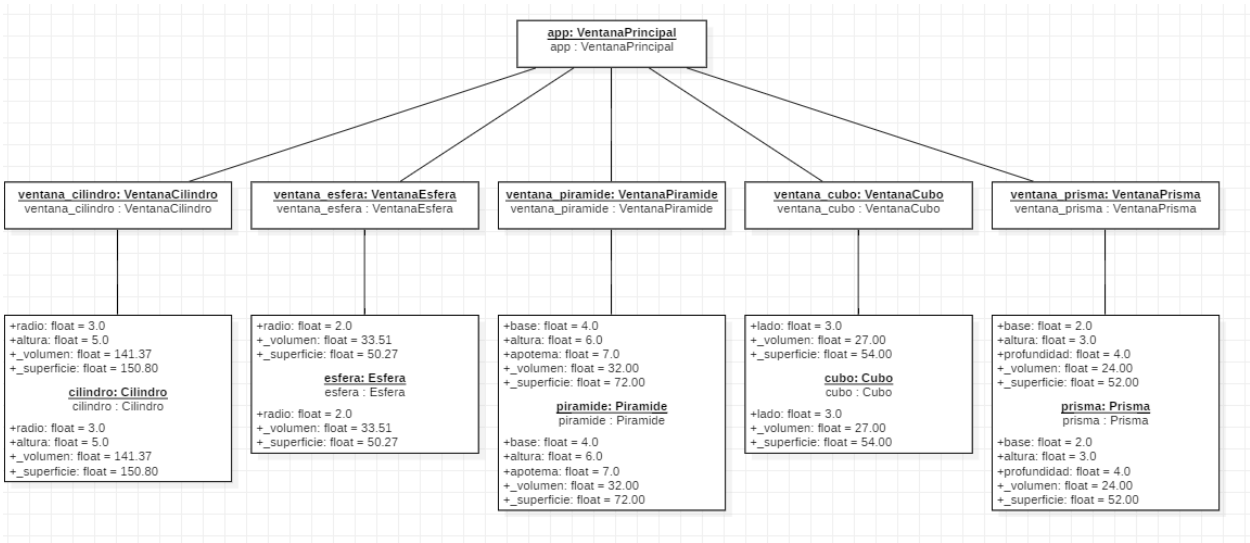
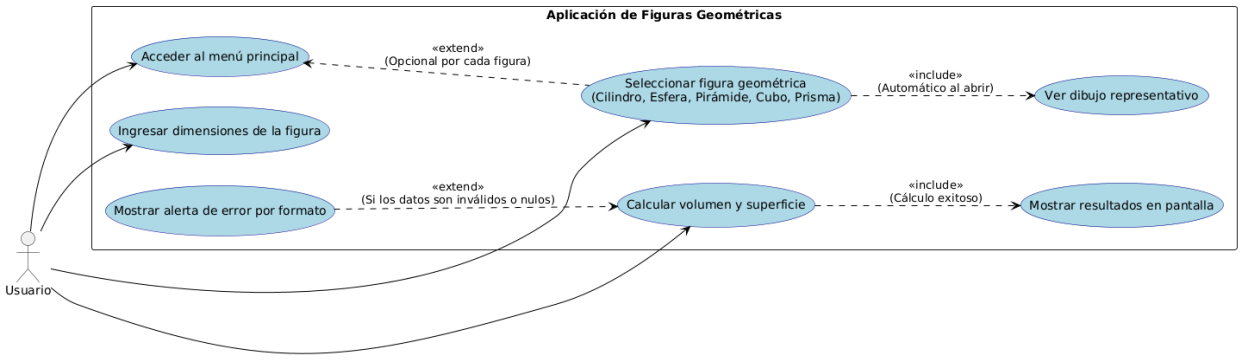


Diagrama de usos:



Url repositorio

https://github.com/jeroberrig/trabajo3-POO-2026-2